

LABORATORIO DE DATOS

Primer Cuatrimestre 2024

Práctica N° 4: Regresión lineal y Cuadrados Mínimos

1. (a) Implementar una función que calcule la pendiente y la ordenada al origen de la recta de regresión lineal con las fórmulas vistas en clase:

$$\beta_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

$$\beta_0 = \bar{y} - \beta_1 \bar{x}$$

donde:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

$$\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$$

La función debe tomar como argumentos a `x` e `y`, que son `pandas.Series` o `numpy.array`, y devolver los valores de β_0 y β_1 .

Sugerencia: recordar que dado un `pandas Series` se utiliza `.mean()` para calcular su promedio y recordar el uso de `np.sum`

```
def coefs_rl(x, y):
    beta_1 = ???
    beta_0 = ???
    return beta_1, beta_0
```

- (b) Con el dataset `gapminder`, utilizar la función implementada en el item anterior para realizar una regresión lineal entre los años y la expectativa de vida en Argentina. Comparar los coeficientes con los obtenidos por `scikit-learn`.

```
datos = gapminder[???]
print(coefs_rl(datos[???], datos[???]))

X = ???
y = ???
modelo = linear_model.LinearRegression(???)
modelo.fit(???, ???)
beta_1 = modelo.???
beta_0 = modelo.???
print(beta_1, beta_0)
```

2. En este ejercicio trabajaremos con el dataset de inmuebles (`inmuebles.csv` en la página de la materia). El dataset contiene datos sobre inmuebles que están a la venta en cierta ciudad: su superficie en m^2 , su precio en millones de pesos y la zona de la ciudad donde se encuentra. Recordar como cargar un dataset desde un `.csv` y visualizar sus primeras filas:

```
datos = pd.read_csv('inmuebles.csv')
datos.head()
```

- (a) Realizar un gráfico de dispersión (scatterplot) que muestre la relación entre la superficie y el precio de cada inmueble.
- (b) Realizar un gráfico de la regresión lineal entre ambas variables. El gráfico debe titularse “Datos inmobiliarios” y la recta de Regresión Lineal debe tener una leyenda que diga “Regresión”.
- (c) Calcular los coeficientes de la recta que mejor ajusta a los datos. Según el modelo, ¿qué podríamos interpretar sobre el costo del metro cuadrado en la ciudad?
- (d) Para medir qué tan bien ajusta la recta a los datos, vamos a implementar dos funciones: una que calcule el error cuadrático medio (ECM) y otra que calcule el coeficiente de determinación R^2 . Recordemos que:

$$\text{Error cuadrático medio: } ECM = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\text{Coeficiente de determinación: } R^2 = \frac{\sum_{i=1}^n (\hat{y}_i - \bar{y})^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Para calcular ambas necesitamos los datos x , y y los coeficientes de la recta.

```
def ecm(x, y, pendiente, o_origen):
    return ???

def r_cuad(x, y, pendiente, o_origen):
    return ???
```

- (e) Utilizando las funciones implementadas en el ítem anterior, calcular el ECM y el R^2 del ajuste realizado en el ítem b). ¿En qué unidades está cada medida? ¿Cómo podemos interpretarlas?
- (f) Comparar los resultados obtenidos en el ítem anterior con los proporcionados por `r2_score` y `mean_squared_error` de `scikit-learn`
- (g) Mediante la confección de un boxplot, decidir en cuál de las zonas hay mayor variabilidad de precios. ¿Hay algún outlier?
- (h) Para cada una de las zonas de la ciudad, calcular los coeficientes, el ECM y R^2 de la recta que mejor aproxima a los datos.

- (i) Graficar los datos y el ajuste lineal de cada zona utilizando el método `facet()` de `Plot()` (recordar ejercicio 5.b de la práctica 3) ¿Cuál es el valor del metro cuadrado en cada zona? ¿Qué podemos concluir si comparamos estos valores con lo obtenido en el ítem c) ?
- (j) Supongamos que queremos poner a la venta un inmueble de 105 m². Sólo con esa información y teniendo en cuenta los items anteriores, ¿cuál sería el precio de referencia para la venta? Si sabemos además que el inmueble está en la Zona 2, ¿cambiaría en algo el valor calculado anteriormente?
- (k) Si me ofrecen un inmueble de 100 m² en la Zona 2 a un precio de 300, ¿qué tan barato o caro es respecto a su precio de referencia?
- (l) *Efecto de los outliers*. En este item trabajaremos con los datos de `inmuebles_outliers.csv`, que tiene los mismos datos que `inmuebles.csv`, salvo cuatro que son outliers.
 - i. Realizar un boxplot que permita identificar en qué zona(s) se encuentran los outliers.
 - ii. Comparar los coeficientes del ajuste lineal de la(s) zona(s) afectada(s) con los obtenidos en el ítem h)

3. Utilizando el dataset `tips` de `seaborn`:

```
datos = sns.load_dataset('tips')
```

realizar la Regresión Lineal donde la variable X es `total_bill` menos el promedio de `total_bill` y la variable Y es `tip`. Responder las siguientes preguntas:

- (a) ¿Qué interpretación se le puede dar a β_0 ? *Pista*: calcular el promedio de las propinas.
 - (b) ¿Cambia el valor de β_1 respecto a la Regresión Lineal de `total_bill` vs. `tip`?
4. En el archivo `bitcoin.csv` se encuentran datos de cotización de Bitcoin desde el 17/09/2014 hasta el 19/02/2022 ¹. Cargamos el dataset:

```
btc = pd.read_csv('datos/bitcoin.csv')
btc.head()
```

Nos interesa analizar la evolución del precio de cierre (*Close*) en periodo comprendido entre el 01/01/2021 y el 01/07/2021:

```
# Nos aseguramos que pandas interprete la fecha correctamente
btc['Date'] = pd.to_datetime(btc['Date'], format='%Y-%m-%d')

# Filtramos el dataset en el periodo de interes
btc_2021 = btc[(btc['Date'] > "2021-01-01") & (btc['Date'] < "2021-07-01")]
```

Visualizar el ajuste lineal para los datos del dataframe `btc_2021`. En este caso, ¿resulta más conveniente un scatterplot o un gráfico de líneas para los datos? ¿Te resultaría útil utilizar esta recta para predecir el valor de BTC o para describir el cambio de su valor en este periodo?

¹Fuente: <https://www.kaggle.com/datasets/meetnagadia/bitcoin-stock-data-sept-17-2014-august-24-2021>

5. En este ejercicio utilizaremos el dataset `healthexp` de `seaborn`, donde se recopila cada año (`Year`) lo que cada país (`Country`) invierte en salud por habitante (`Spending_USD`) y su expectativa de vida (`Life_Expectancy`).

Nos enfocaremos en los datos de Japón, nuestra variable predictora será `Spending_USD` y la dependiente será `Life_Expectancy`.

- (a) Visualizar en un mismo gráfico los datos y los polinomios de grado 1, de grado 2 y de grado 3 que mejor ajustan a los datos. Añadir etiquetas que para facilitar la interpretación del gráfico.
 - (b) En base al gráfico obtenido en el ítem anterior, elegir el grado que consideras que mejor ajusta a los datos. Utilizando `scikit-learn`, calcular los coeficientes de ese polinomio.
 - (c) Calcular el R^2 y el ECM.
 - (d) Según el polinomio obtenido en el ítem anterior, estimar cuál sería la expectativa de vida de los habitantes de Japón si el país invirtiera USD 5000.
 - (e) Visualizar el polinomio de grado 10 que mejor ajusta a los datos. ¿Se aprecia una mejora? ¿Resulta conveniente ajustar con un polinomio de grado 50?
6. [Opcional] En este ejercicio, implementaremos una función que calcule la Media Móvil de Cuadrados Mínimos (LSMA, por sus siglas en inglés) y lo aplicaremos a los datos de cotización de Bitcoin durante los primeros seis meses de 2021.

- (a) Completar el siguiente código para implementar la función que calcula los puntos de la curva de LSMA. La función toma como argumentos a `x` e `y`, que son `pandas.Series` o `numpy.array`, y a `k` que es la longitud de la ventana ($k \geq 2$):

```
def lsma(x, y, k):
    # Los valores para los cuales se calculan los puntos de la
    # curva de LSMA
    x_approx = np.array([i for i in range(???, len(x))])

    # Las coordenadas 'y' de los puntos de la curva de LSMA
    y_approx = []
    for i in x_approx:
        # Calculamos la recta de regresion para los k datos previos
        .

        # Usamos la funcion del Ejercicio 1a.
        beta_1, beta_0 = coefs_rl(x[???: i], y[???: i])
        # Agregamos a y_approx la predicción de la recta para i
        y_approx.append(???)

    return x_approx, np.array(y_approx)
```

- (b) Utilizando el dataset `bitcoin.csv`, graficar los datos correspondientes al periodo comprendido entre el 01/01/2021 y el 01/07/2021, inclusive, junto con las curvas de LSMA con ventana de 9 y de 25 días. El eje x del gráfico debe ser la cantidad de días que transcurrieron desde el 01/01/2021. Consideraremos el precio de cierre de cada día (es decir, la columna `Close`). Puede utilizar el siguiente código como ayuda:

```
btc = pd.read_csv('datos/bitcoin.csv')
# Indicamos a pandas que el tipo de la columna Date es fecha
```

```

btc['Date'] = pd.to_datetime(btc['Date'], format='%Y-%m-%d')

# Nos quedamos con los datos requeridos, reindexamos y agregamos
# una columna Day con la cantidad de días que pasaron desde el
# inicio del periodo
btc_2021 = (btc[(btc['Date'] >= "2021-01-01") & (btc['Date'] <= "
2021-07-01")])
            .reset_index(drop=True)
            .reset_index(names='Day'))

# Aplicamos la función del ítem anterior para obtener los puntos de
# las curvas de LSMA
x_lsma_24, y_lsma_24 = lsma(???, ???, ???)

x_lsma_9, y_lsma_9 = lsma(???, ???, ???)

# Graficamos
plot = (
    so.Plot()
    # La curva con los datos de la cotización
    .add(so.Line(color='grey'), x=???, y=???)
    # La curva de LSMA de 24 días
    .add(so.Line(linewidth=0.8), x=???, y=???, label='LSMA(24)')
    # La curva de LSMA de 9 días
    .add(so.Line(color='red', linewidth=0.8), x=???, y=???, label='
LSMA(9)')
    .label()
)

```